

# CLOUD-BASED LINUX SECURITY MONITORING LAB

Wazuh Cloud + Kali Linux + VirtualBox

---

## PROJECT REPORT

---

Name: Lajin C P

Date: 08/09/2026

### Tools Used

Wazuh Cloud

Wazuh Agent 4.14.7

Kali Linux 2024.3

VirtualBox

## Project at a Glance

---

| Capability                | Result                                  |
|---------------------------|-----------------------------------------|
| Wazuh Agent Connection    | Successful                              |
| Vulnerability Detection   | Successful                              |
| SSH Monitoring            | Successful                              |
| File Integrity Monitoring | Successful                              |
| Custom Detection Rule     | Attempted / requires further validation |

## Executive Summary

---

This report documents a small, self-contained cloud-based Linux security monitoring lab built to demonstrate practical Security Operations Center (SOC) skills. A Kali Linux virtual machine, hosted in VirtualBox, was configured as a monitored endpoint using the Wazuh Agent and connected to a Wazuh Cloud environment for centralized log collection, alerting, and analysis.

The project successfully demonstrated the following core capabilities:

- Endpoint deployment and telemetry collection through the Wazuh Agent
- Vulnerability detection on the monitored endpoint
- SSH authentication event detection
- File Integrity Monitoring (FIM) of a critical Linux directory
- Structured security event investigation using Wazuh Cloud's dashboards

A series of controlled security events — including a failed SSH login using a nonexistent username and a monitored configuration-file modification — were deliberately generated in the lab to validate that the monitoring pipeline was functioning end-to-end. In addition, an attempt was made to build a custom Wazuh detection rule; this attempt did not fully succeed and is documented transparently as a troubleshooting exercise later in this report.

## Introduction

---

Wazuh is an open-source security monitoring platform that combines endpoint detection, log analysis, vulnerability detection, and file integrity monitoring into a single solution. Agents installed on monitored systems forward security-relevant telemetry to a central manager (in this case, Wazuh Cloud), where the data is indexed, correlated against detection rules, and presented through investigative dashboards.

Endpoint security monitoring is a foundational SOC capability. Without visibility into what is happening on individual hosts — authentication attempts, software vulnerabilities, and changes to sensitive files — an organization has no reliable way to detect misuse, misconfiguration, or compromise.

This lab was built as a small, controlled security monitoring laboratory, not a production SOC deployment. Its purpose was to gain hands-on experience deploying an agent-based monitoring solution, generating and investigating security telemetry, and practicing the workflow a junior SOC analyst would follow when triaging alerts.

The basic workflow used throughout the lab was:

- Kali Linux endpoint (SSH service, system logs, file system, installed software)
- Wazuh Agent (collects and forwards telemetry)
- Wazuh Cloud (indexing, detection rules, dashboards)
- Security alerts / vulnerability information / FIM events
- Investigation (SOC analyst review)
- Response recommendations

## Objectives

---

1. Deploy Wazuh Agent on a Linux endpoint.
2. Connect the endpoint to Wazuh Cloud.
3. Verify endpoint telemetry.
4. Detect known software vulnerabilities.
5. Monitor SSH authentication activity.
6. Simulate controlled failed SSH login attempts.
7. Monitor important Linux directories using FIM.
8. Generate a controlled file modification event.
9. Investigate detected security events.
10. Attempt to create a custom Wazuh detection rule.
11. Document findings and response recommendations.

## Lab Environment

---

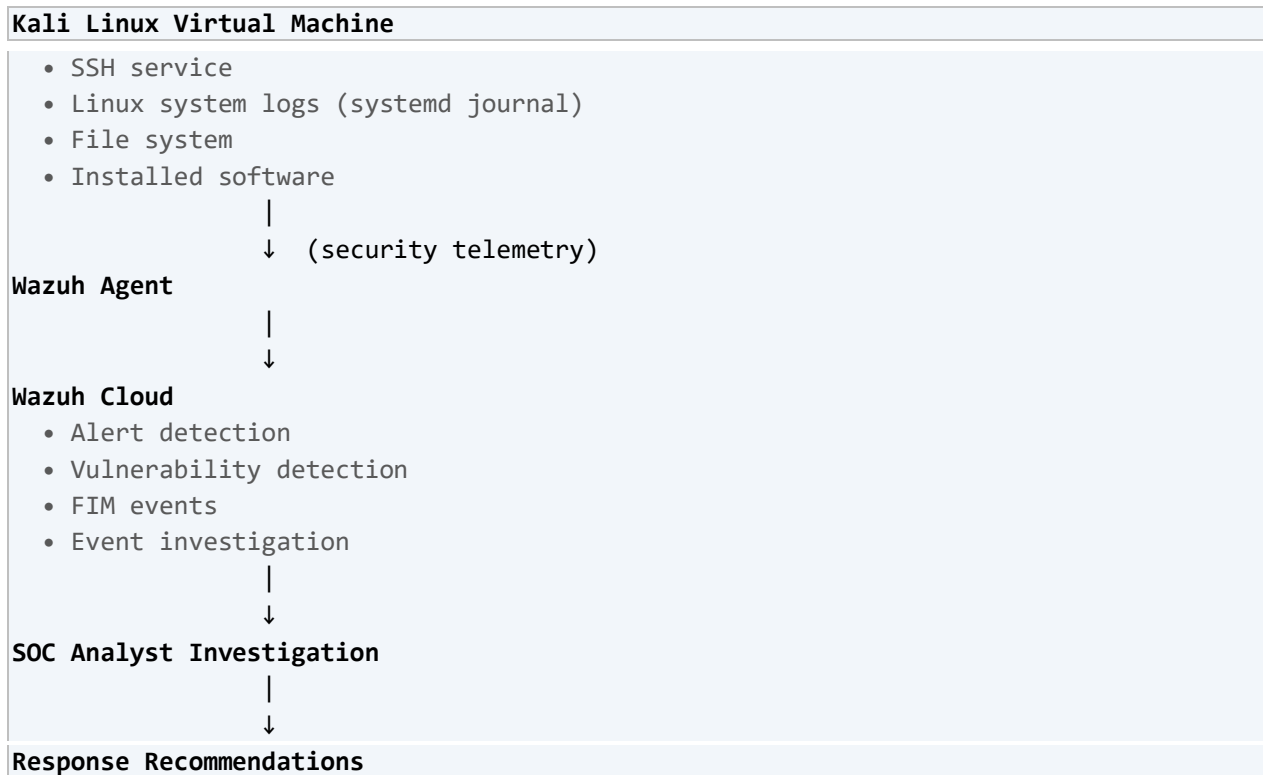
The table below summarizes the components used to build the lab, based on the configuration confirmed in the Wazuh Cloud dashboard and Kali Linux terminal.

| Component         | Details            |
|-------------------|--------------------|
| Security Platform | Wazuh Cloud        |
| Endpoint          | Kali Linux 2024.3  |
| Agent             | Wazuh Agent 4.14.7 |

| Component           | Details                                                                |
|---------------------|------------------------------------------------------------------------|
| Virtualization      | VirtualBox                                                             |
| Agent Name          | kali-lab                                                               |
| Agent ID            | 001                                                                    |
| Endpoint IP Address | 10.0.3.15                                                              |
| Wazuh Cluster Node  | wazuh-manager-master-0                                                 |
| Monitoring          | SSH authentication, File Integrity Monitoring, Vulnerability Detection |
| Log Collection      | Linux systemd journal                                                  |

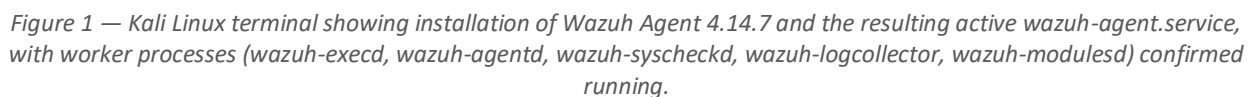
## Architecture

The lab followed a straightforward agent-to-cloud monitoring architecture, illustrated below.



Each layer has a distinct role: the Kali Linux endpoint generates raw activity; the Wazuh Agent collects and forwards that activity as structured telemetry; Wazuh Cloud applies detection rules, indexes the data, and surfaces vulnerability and integrity information; and the SOC analyst layer investigates flagged events and produces response recommendations.

## Wazuh Agent Deployment



Successful registration and endpoint telemetry were then confirmed directly from the Wazuh Cloud Endpoints page:



*Figure 2 — Kali Linux endpoint successfully connected to Wazuh Cloud.*

The Endpoints dashboard confirms one active agent, running Kali GNU/Linux 2024.3, in the default group. The agent inventory table lists the following verified details:

| Field            | Value                  |
|------------------|------------------------|
| Agent ID         | 001                    |
| Agent Name       | kali-lab               |
| IP Address       | 10.0.3.15              |
| Group            | default                |
| Operating System | Kali GNU/Linux 2024.3  |
| Cluster Node     | wazuh-manager-master-0 |
| Agent Version    | v4.14.7                |
| Status           | active                 |

This confirms the endpoint was actively communicating with Wazuh Cloud and forwarding telemetry for analysis.

## Vulnerability Detection

---

Vulnerability detection demonstrates a SOC's ability to identify known, exploitable weaknesses in software running on monitored endpoints before those weaknesses are used against the organization. Wazuh Cloud continuously evaluates installed packages against known vulnerability feeds and surfaces matches by severity.

During the lab, Wazuh Cloud identified one confirmed critical-severity vulnerability on the kali-lab endpoint:

| Field           | Result         |
|-----------------|----------------|
| CVE             | CVE-2025-69872 |
| Package         | diskcache      |
| Severity        | Critical       |
| CVSS Base Score | 9.8            |
| Endpoint        | kali-lab       |

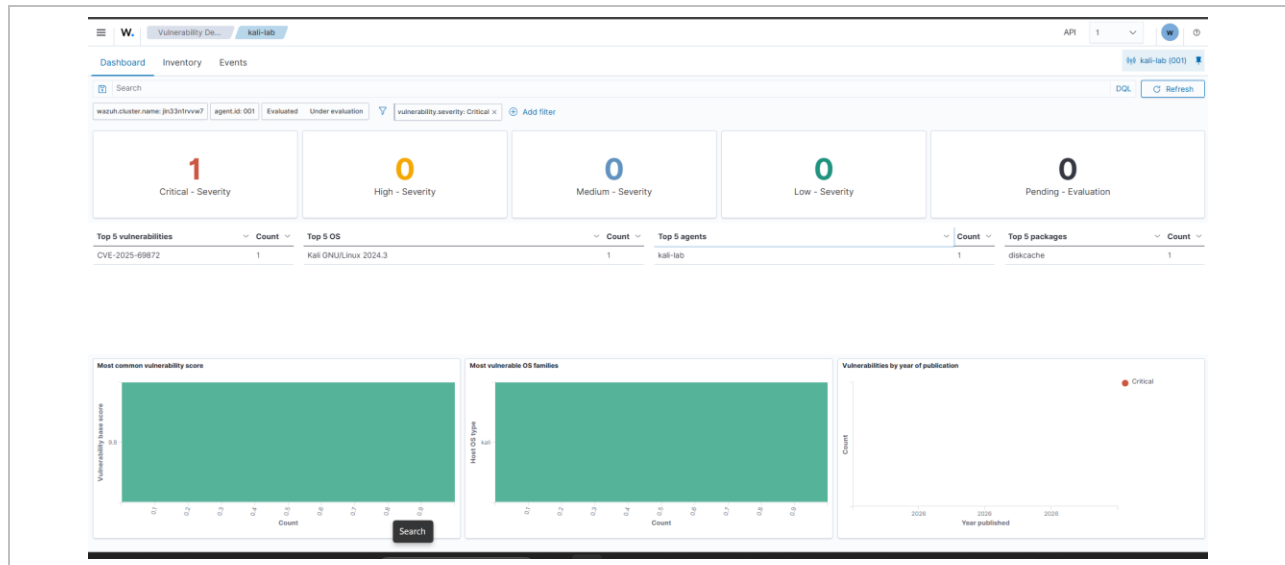


Figure 3 — Wazuh Cloud vulnerability detection identifying a critical vulnerability on the monitored Kali Linux endpoint.

The vulnerability dashboard, filtered to agent.id 001 with Critical severity, confirms exactly one Critical finding on kali-lab: CVE-2025-69872 in the diskcache package, with a base score of 9.8 and zero findings at High, Medium, or Low severity. This finding was not exploited during the lab and no remediation was performed as part of this exercise.

From a security operations perspective, vulnerability findings should be triaged and prioritized based on severity, exploitability, the exposure of the affected system, and the potential business impact of the affected asset. A CVSS score of 9.8 places this finding in the highest priority band and would normally warrant immediate investigation and patching in a production environment.

## SSH Authentication Monitoring

SSH authentication monitoring was tested in the controlled Kali Linux environment to confirm that Wazuh could detect and classify login activity on the endpoint. Before testing, the SSH service status was verified as active and listening on port 22:

```

File Edit Shell View Help
(kali@kali)~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-08 21:22:07 IST; 52min ago
     Invocation: 92e015adf3c74f7c8cd8f900cf743b87
       Docs: man:sshd(8)
             man:sshd_config(5)
    Main PID: 672 (sshd)
      Tasks: 1 (limit: 2591)
     Memory: 2.3M (peak: 2.6M)
        CPU: 100ms
      CGroup: /system.slice/ssh.service
             └─673 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 08 21:22:06 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Sep 08 21:22:07 kali sshd[673]: Server listening on 0.0.0.0 port 22.
Sep 08 21:22:07 kali sshd[673]: Server listening on :: port 22.
Sep 08 21:22:07 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)~$ ssh fakeuser@localhost
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
fakeuser@localhost: Permission denied (publickey,password).

(kali@kali)~$ sudo tail -n 20 /var/log/auth.log
tail: cannot open '/var/log/auth.log' for reading: No such file or directory

(kali@kali)~$

```

Figure 4 — Kali Linux terminal confirming the SSH service is active and showing a controlled login attempt using the nonexistent username “fakeuser.”

A controlled test was then performed using a nonexistent username to generate a failed SSH authentication event, without attempting to access any real account:

```
ssh fakeuser@localhost
```

The terminal output confirms three consecutive “Permission denied, please try again” responses followed by a final “Permission denied (publickey,password)” message — consistent with an authentication attempt against a username that does not exist on the system.

This event was successfully detected by Wazuh. The following event details were confirmed as part of the lab:

| Field                           | Value                                            |
|---------------------------------|--------------------------------------------------|
| Agent                           | kali-lab                                         |
| Agent ID                        | 001                                              |
| Username                        | fakeuser                                         |
| Source IP                       | ::1                                              |
| Wazuh Rule ID                   | 5710 (built-in)                                  |
| Rule Level                      | 5                                                |
| Rule Description                | sshd: Attempt to login using a non-existent user |
| Related MITRE ATT&CK Techniques | T1110.001, T1021.004                             |

Broader authentication activity on the endpoint was also visible from the Wazuh Cloud Threat Hunting dashboard for kali-lab, shown below:

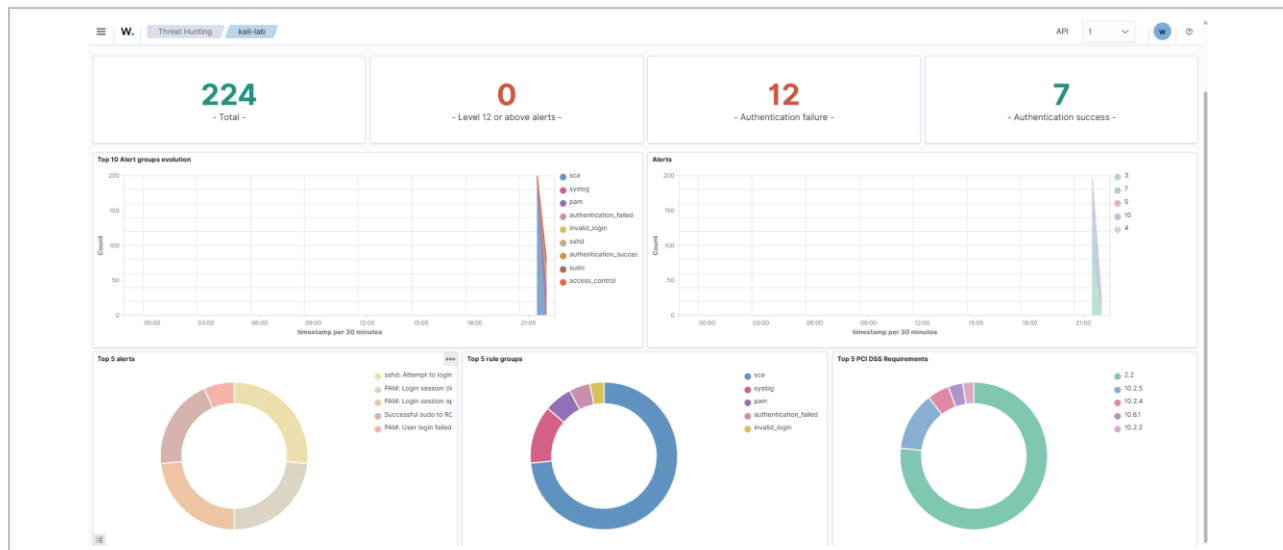


Figure 5 — Wazuh Cloud Threat Hunting dashboard for kali-lab showing authentication-related alert volume and categorization.

Over the observed window, the dashboard recorded 224 total alerts for the endpoint, including 12 authentication failures and 7 authentication successes, with zero alerts at severity level 12 or above. The top alert groups included sshd, syslog, pam, authentication\_failed, and invalid\_login, and the top individual alerts included “sshd: Attempt to login...” and several PAM session and login-failure alerts — consistent with the sshd rule group that generates the nonexistent-user detection described above.

It is important to distinguish between related but distinct SSH authentication concepts:

- Authentication failure — a login attempt was rejected (wrong password, disabled account, etc.), but the username may be valid.
- Nonexistent user — the login attempt referenced a username that does not exist on the system at all, which is the specific condition rule 5710 detects.
- Successful authentication — the credentials presented were valid and access was granted.
- Repeated authentication attempts — multiple failures in a short window from the same source, which can indicate password guessing or brute-force behavior and typically warrants closer investigation.

This test was a controlled validation exercise performed by the lab operator and does not represent a real attack. It confirms that the built-in Wazuh SSH detection rules were functioning correctly against live telemetry from the endpoint.

## File Integrity Monitoring

File Integrity Monitoring (FIM) tracks changes to specified files and directories, alerting when content, permissions, ownership, or other attributes are modified. This capability is valuable for detecting unauthorized configuration changes, persistence mechanisms, or tampering with system files.

The lab was configured to monitor the following important Linux directories:

- /etc
- /usr/bin
- /usr/sbin
- /bin
- /sbin
- /boot

Real-time monitoring was enabled for /etc during the test. A controlled test file, /etc/wazuh-fim-test.txt, was created and then modified to generate a FIM event.

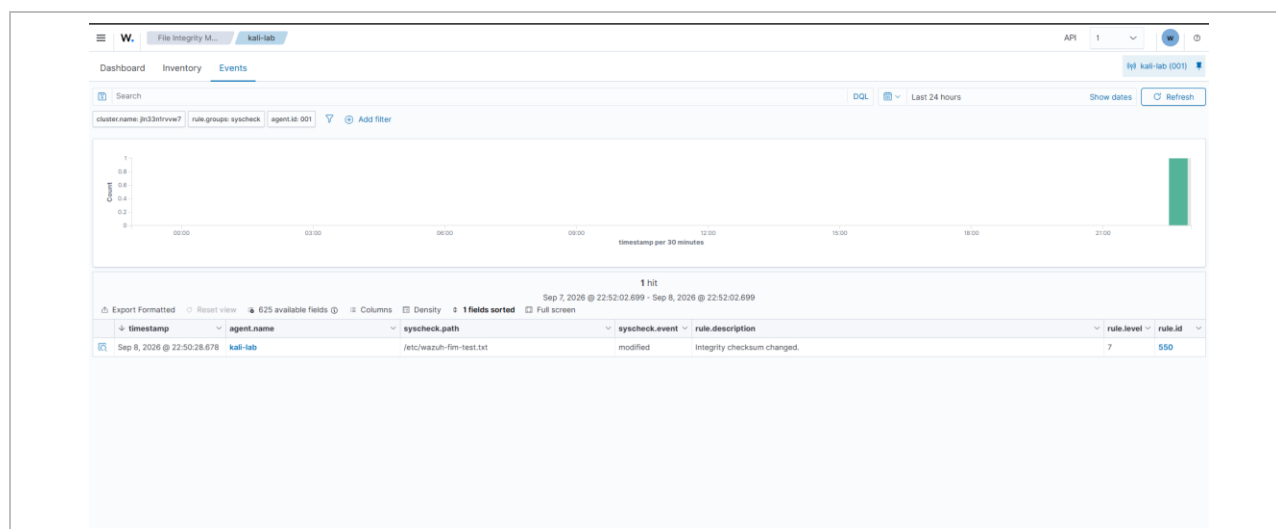


Figure 6 — Wazuh File Integrity Monitoring detecting a controlled modification to a monitored file.

The FIM Events view, filtered to agent.id 001 and the syscheck rule group, returned a single confirmed hit for the test window:

| Field                       | Value                      |
|-----------------------------|----------------------------|
| Timestamp                   | Sep 8, 2026 @ 22:50:28.678 |
| Agent                       | kali-lab                   |
| File Path (syscheck.path)   | /etc/wazuh-fim-test.txt    |
| Event Type (syscheck.event) | modified                   |
| Rule ID                     | 550                        |
| Rule Level                  | 7                          |

| Field            | Value                       |
|------------------|-----------------------------|
| Rule Description | Integrity checksum changed. |

Monitoring /etc is particularly valuable from a defensive standpoint because it holds core system and service configuration files. Unauthorized modification of files in this directory can be an indicator of compromise, an attempt at persistence, privilege escalation, or general system tampering, depending on the surrounding context and what specifically changed.

This particular modification was an authorized laboratory test performed by the lab operator to validate that real-time FIM was functioning correctly, and it was successfully detected end-to-end by Wazuh.

## Security Event Investigation

---

A SOC analyst investigating each of the events generated in this lab would review the following data points.

### SSH Authentication Event

- Timestamp of the attempt
- Source IP address (:::1 in this controlled test)
- Username used in the attempt (fakeuser)
- Number of attempts from the same source
- Authentication result (failed)
- Rule ID and severity (5710, level 5)
- Full raw log entry
- Any related authentication events from the same host or source

### Vulnerability Finding

- CVE identifier (CVE-2025-69872)
- Affected package (diskcache)
- Severity (Critical)
- CVSS score (9.8)
- Affected endpoint (kali-lab)
- Remediation status (not yet remediated as of this report)

### File Integrity Event

- File path (/etc/wazuh-fim-test.txt)
- Event type (modified)
- Timestamp (Sep 8, 2026 @ 22:50:28.678)
- Responsible user/process, where available
- Whether the change was authorized (confirmed authorized test)

- Any related file or system events

The three events generated during the lab are summarized in the investigation matrix below.

| Event                  | Detection      | Severity | Investigation Focus                                             |
|------------------------|----------------|----------|-----------------------------------------------------------------|
| SSH nonexistent user   | Rule 5710      | 5        | Review source IP, username, frequency, and related login events |
| Critical vulnerability | CVE-2025-69872 | Critical | Assess affected package and determine remediation path          |
| File modification      | Rule 550       | 7        | Verify whether the modification was authorized                  |

It is important to note that severity values shown above are assigned by Wazuh's rule set and reflect the potential significance of an event type — they do not, by themselves, mean an event is malicious. Each event still requires human review and context before a determination is made.

## Detection Analysis

---

### Visibility

Wazuh provided centralized visibility into endpoint security telemetry for the kali-lab agent, consolidating authentication activity, vulnerability data, and file integrity events into a single set of dashboards.

### Detection

Wazuh successfully detected all three categories of activity generated during the lab:

- SSH authentication activity, including a failed login using a nonexistent user
- Vulnerability information, including one critical-severity CVE
- File integrity changes to a monitored configuration directory

### Investigation

Each detected event carried metadata useful for investigation, including agent identity, timestamp, rule ID, rule severity, source IP, username, the full raw log, and (for FIM) the affected file path.

### Security Operations Value

Together, these capabilities support a SOC analyst in identifying suspicious authentication behavior (repeated failures, nonexistent-user attempts), prioritizing vulnerabilities for remediation based on severity and CVSS score, and investigating unauthorized or unexpected file changes on critical systems.

## Response Recommendations

---

The following recommendations reflect standard SOC practice for the event types observed in this lab. No remediation actions were performed as part of this exercise unless explicitly noted elsewhere in this report.

## SSH

- Investigate any repeated authentication failures from the same source.
- Identify and, where necessary, block or rate-limit suspicious source IP addresses.
- Review whether any successful logins followed a series of failures.
- Confirm whether attempted usernames correspond to legitimate accounts.
- Consider SSH hardening measures (key-based authentication, disabling root login, restricting source networks).
- Apply rate limiting or other brute-force controls where appropriate.

## Vulnerability

- Identify the affected package and installed version.
- Check for an available security update or patched package version.
- Update or remove the vulnerable package where appropriate.
- Re-scan the endpoint after remediation to confirm the finding is resolved.
- Prioritize Critical and High severity findings ahead of lower-severity items.

## File Integrity

- Determine whether the detected change was authorized.
- Identify the responsible user or process where possible.
- Compare the change against any approved change records.
- Investigate related security events around the same time window.
- Restore the original file if the modification is found to be unauthorized.

## Custom Detection Rule Attempt

---

As part of this project, a custom Wazuh detection rule was attempted for the SSH nonexistent-user event, intended to layer a higher-severity, purpose-built alert on top of the existing built-in detection.

| Field             | Value                                                |
|-------------------|------------------------------------------------------|
| Parent Rule       | 5710                                                 |
| Custom Rule ID    | 100001                                               |
| Intended Severity | 10                                                   |
| Intended Purpose  | Identify SSH login attempts using a nonexistent user |

During validation testing, the default Wazuh rule 5710 continued to generate the expected event as designed; however, the custom rule 100001 did not appear in Wazuh Cloud Discover. As a result, this report does not claim that the custom rule was successfully deployed. Instead, the custom rule development is documented here as an attempted enhancement that did not produce the expected alert during validation, and as a troubleshooting exercise for future iterations of the lab.

Areas that would normally be investigated further to resolve this include:

- Confirming the custom rule file was deployed to the correct location and picked up by the manager
- Verifying the rule file loaded without syntax errors
- Confirming the manager/ruleset reload completed successfully
- Reviewing the parent-rule relationship and match conditions between rule 5710 and rule 100001
- Re-checking rule syntax against the Wazuh rule schema
- Considering how cloud-side rule processing in a managed Wazuh Cloud environment may differ from a self-hosted manager
- Confirming that the triggering event actually matched the custom rule's field conditions

Importantly, the underlying SSH detection itself was successful throughout — the built-in rule 5710 reliably detected the nonexistent-user login attempt. This section reflects an enhancement attempt on top of already-working detection, not a failure of the core monitoring pipeline.

## Challenges and Troubleshooting

Several practical challenges were encountered and worked through during the lab, consistent with real-world security engineering work.

- Linux authentication events on this Kali Linux system were collected through the systemd journal rather than a traditional `/var/log/auth.log` file. Attempting to tail `/var/log/auth.log` directly returned “No such file or directory,” confirming that the distribution routes authentication logging through journald rather than a flat log file.
- The Wazuh Agent's own log (`ossec.log`) was reviewed and confirmed the agent was monitoring journal entries directly, shown below.

```
[kali@kali]:~$ sudo cat /var/ossec/logs/ossec.log
2026/09/08 21:58:02 wazuh-logcollector: INFO: Monitoring output of command[0]: df -P
2026/09/08 21:58:02 wazuh-logcollector: INFO: Monitoring full output of command[0]: netstat -tlnp | sed 's/^([[:alnum:]]\+)\ \+([[:digit:]]\+)\ \+([[:digit:]]\+)\ \+(\.\.?)\+([[:digit:]]\+)\ \+\{[-@-]\+\}\ \+\.\ \+[[:digit:]]\+\|fail\nm};\|\.\|\.\|\.\ = \| = \| sort -k 4,g | sed 's/^(\.\.\.)=\\|/\| sed 1,2d'
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/log/nginx/access.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/log/nginx/error.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/log/apache2/error.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/log/apache2/access.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/ossec/logs/active-responses.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Analyzing file: /var/log/hpkg.log'.
2026/09/08 21:58:02 wazuh-logcollector: INFO: Started (pid: 2284).
2026/09/08 21:58:02 wazuh-modulesd: INFO: Started (pid: 2269).
2026/09/08 21:58:02 wazuh-modulesd:agent-upgrade: INFO: (6151): Module Agent Upgrade started.
2026/09/08 21:58:02 wazuh-modulesd:iscat: INFO: Module disabled. Exiting...
2026/09/08 21:58:02 wazuh-modulesd:osquery: INFO: Module disabled. Exiting...
2026/09/08 21:58:02 sca: IRM Module started.
2026/09/08 21:58:02 sca: INFO: Loaded policy '/var/ossec/ruleset/sca/sca_distro_independent_linux.yml'
2026/09/08 21:58:02 sca: INFO: Starting control thread.
2026/09/08 21:58:02 sca: INFO: Starting Security Configuration Assessment scan.
2026/09/08 21:58:02 wazuh-modulesd:syccollector: INFO: Module started.
2026/09/08 21:58:02 wazuh-modulesd:syccollector: INFO: Starting evaluation.
2026/09/08 21:58:02 sca: INFO: Starting evaluation of policy: '/var/ossec/ruleset/sca/sca_distro_independent_linux.yml'
2026/09/08 21:58:02 wazuh-modulesd:syccollector: INFO: Evaluation finished.
2026/09/08 21:58:24 wazuh-logcollector: INFO: (9283): Monitoring journal entries.
2026/09/08 21:58:25 wazuh-agentd: INFO: Syscheck [18] User defined signal 3) Received. Reload agents.
2026/09/08 21:58:25 wazuh-agentd: INFO: Buffer agent.conf updated, enable 1 size: 5000
2026/09/08 21:58:34 wazuh-syscheckd: INFO: (4869): File integrity monitoring scan ended.
2026/09/08 21:58:34 wazuh-syscheckd: INFO: FIM sync module started.
2026/09/08 21:58:40 sca: INFO: Evaluation finished for policy: '/var/ossec/ruleset/sca/sca_distro_independent_linux.yml'
2026/09/08 21:59:08 sca: INFO: Security Configuration Assessment scan finished. Duration: 18 seconds.
2026/09/08 21:59:29 routecheck: INFO: Ending routecheck scan.

[kali@kali]:~$
```

Figure 7 — Wazuh Agent log (*ossec.log*) confirming journal-based log collection and the file integrity monitoring scan lifecycle.

The log excerpt above confirms several relevant details: the log collector explicitly reports “Monitoring journal entries,” the agent configuration was reloaded after a signal was sent (consistent with applying an updated *agent.conf*), and the syscheck (FIM) module completed a scan and started its sync module — supporting the real-time /etc monitoring used in the FIM test.

- File Integrity Monitoring initially used scheduled (periodic) monitoring; real-time monitoring was configured for /etc specifically to support the controlled FIM test performed in this lab.
- The custom Wazuh detection rule (100001) was created and the ruleset was reloaded, but it did not generate the expected custom alert during validation, as documented in the previous section.
- The built-in SSH rule 5710 successfully detected the test event throughout, confirming the core detection pipeline was working correctly even while the custom rule was still being debugged.

Testing and troubleshooting are important parts of security engineering, because detection logic must be validated against real telemetry rather than assumed to work correctly once written. The issues encountered here — journal-based logging instead of a flat log file, and an unvalidated custom rule — are exactly the kind of findings a security engineer documents and iterates on rather than treats as project failures.

## Conclusion

---

This project successfully demonstrated a small, cloud-based Linux security monitoring environment built around Wazuh Cloud and a Kali Linux endpoint. The following components were implemented and validated:

- Wazuh Agent deployment on the Kali Linux endpoint
- Wazuh Cloud integration and confirmed endpoint telemetry
- Vulnerability detection, identifying one critical-severity CVE
- SSH authentication monitoring, including detection of a controlled nonexistent-user login attempt
- File Integrity Monitoring, including detection of a controlled configuration file modification
- Structured security event investigation across all three event types

A custom detection rule was also attempted and is documented as an unresolved troubleshooting item rather than a completed feature. Taken as a whole, the lab reflects a realistic, appropriately-scoped exercise in endpoint security monitoring rather than a claim of production-grade SOC capability.

## Future Improvements

---

12. Add more Linux endpoints to the monitored environment.
13. Create and fully validate additional custom detection rules.
14. Add brute-force detection scenarios with repeated authentication failures.
15. Monitor privilege escalation activity (e.g., sudo usage patterns).
16. Integrate automated response actions for high-severity alerts.
17. Build dedicated dashboards for authentication and FIM events.

18. Add alert correlation across multiple event types.
19. Perform vulnerability remediation and re-scan to confirm resolution.
20. Add Windows endpoint monitoring for cross-platform coverage.
21. Build a larger, multi-host SOC-style lab environment.

## Evidence / Screenshot Index

The following figures were captured directly from the Wazuh Cloud dashboard and the Kali Linux terminal during the lab and are referenced throughout this report.

| Figure   | Description                                                                      | Section                        |
|----------|----------------------------------------------------------------------------------|--------------------------------|
| Figure 1 | Kali Linux terminal — Wazuh Agent installation and active service status         | Wazuh Agent Deployment         |
| Figure 2 | Wazuh Cloud Endpoints dashboard — agent kali-lab connected and active            | Wazuh Agent Deployment         |
| Figure 3 | Wazuh Cloud Vulnerability Detection dashboard — CVE-2025-69872                   | Vulnerability Detection        |
| Figure 4 | Kali Linux terminal — SSH service status and controlled fakeuser login attempt   | SSH Authentication Monitoring  |
| Figure 5 | Wazuh Cloud Threat Hunting dashboard — authentication alert summary for kali-lab | SSH Authentication Monitoring  |
| Figure 6 | Wazuh Cloud FIM Events — detected modification of /etc/wazuh-fim-test.txt        | File Integrity Monitoring      |
| Figure 7 | Wazuh Agent ossec.log — journal monitoring and FIM scan lifecycle                | Challenges and Troubleshooting |

**Note on redaction:** The original installation screenshot (used as Figure 1) contains a Wazuh registration password in plain text. That password has been redacted in this report and should also be redacted or cropped from the original screenshot file before it is published, shared, or stored anywhere outside this controlled lab environment.